Introduction to Programming & Python

Programming is the process of giving instructions to a computer to perform specific tasks. Python is a high-level, versatile, and beginner-friendly programming language widely used in various fields such as web development, data analysis, artificial intelligence, and more.

Why Learn Python?

- Easy to learn and use
- Large community support
- Extensive libraries and frameworks
- Cross-platform compatibility

Writing Your First Python Program

To write a Python program, you need a code editor or an Integrated Development Environment (IDE) like VS Code, PyCharm, or simply use an online Python compiler.

Example:

```
# This is a simple Python program

print("Hello, World!")
```

Explanation: The print() function is used to display text or output on the screen.

Python Syntax and Rules

1. **Case Sensitivity:** Python is case-sensitive.
2. **Indentation:** Python uses indentation to define blocks of code.
3. **Comments:** Use # for single-line comments and ''' or """ for multi-line comments.

Example:

```
# Single-line comment
"""
This is a
multi-line comment
"""
```

Variables and Data Types

Variables store data values. Python automatically determines the data type based on the value assigned.

Example:

```
name = "Alice"  # String
age = 25        # Integer
height = 5.6    # Float
is_student = True  # Boolean
```

Common Data Types:

- **int:** Integer numbers
- **float:** Decimal numbers
- **str:** Strings (text)
- **bool:** Boolean (True or False)

Receiving User Input

The input() function is used to receive input from the user.

Example:

```
name = input("What is your name? ")
print("Hello, " + name + "!")
```

Basic Arithmetic Operations

Python supports basic arithmetic operations like addition, subtraction, multiplication, and division.

Example:

```
a = 10

b = 5


# Addition

print(a + b)


# Subtraction

print(a - b)


# Multiplication

print(a * b)


# Division

print(a / b)
```

STRINGS AND STRING OPERATIONS

Strings are sequences of characters. Python provides various methods to manipulate strings.

Example:

```
message = "Hello, Python!"

print(message.upper())  # Converts to uppercase
```

CODEANT

```
print(message.lower())  # Converts to lowercase
print(message.replace("Python", "World"))  # Replaces a substring
```

CONDITIONS: IF, ELSE

Conditional statements allow the execution of different code blocks based on conditions.

Example:

```
age = 18
if age >= 18:
    print("You are an adult.")
else:
    print("You are not an adult.")
```

CONDITIONS: ELIF

The elif statement allows for multiple conditions to be checked in a sequence.

Example:

```
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 75:
    print("Grade: B")
else:
    print("Grade: C")
```

CODEANT

INTRODUCTION TO LOOPS (WHILE)

A while loop continues to execute a block of code as long as a specified condition is true.

Example:

```
count = 1

while count <= 5:

  print("Count:", count)

  count += 1
```

Loops with for

A for loop is used to iterate over a sequence (like a list, tuple, or string).

Example:

```
for i in range(5):

  print("Iteration:", i)
```



MINI PROJECT: SIMPLE CALCULATOR

Create a simple calculator that performs basic arithmetic operations based on user input.

Example:

```
# Simple Calculator

print("Select operation:")

print("1. Addition")

print("2. Subtraction")

print("3. Multiplication")

print("4. Division")


choice = input("Enter choice (1/2/3/4): ")


num1 = float(input("Enter first number: "))

num2 = float(input("Enter second number: "))


if choice == '1':

   print("Result:", num1 + num2)

elif choice == '2':

   print("Result:", num1 - num2)

elif choice == '3':

   print("Result:", num1 * num2)
```

CODEANT

```
elif choice == '4':
    print("Result:", num1 / num2)
else:
    print("Invalid input")
```

